

* Stack Folder

1) StackDemo.java

Entered:- Expression = "{ [()] }" a variable

calling :- StackDemo.isBalanced

Creating object of StackDemo class

So,

Stack = [] [] [] [] ... (100 slots) top = -1
(empty)

using loop :- For loop

For (i = 0 ; i < expression.length(); i++)
length = 6

Inside loop making char variable ch, storing value of expression at index i

i	ch	operation	Stack (bottom-top)	Result
0	{	Push	{	-
1	[	push	{ [	-
2	(	Push	{ [(	-
3	)	Pop → open = (, matches)	{ [	match
4	]	pop → open = [, matches]	{	match
5	}	Pop → open = {, matches }	-	match

Date _____
Page _____

For loop ends ($i=6$, condition false)
return isEmpty() $\rightarrow$ stack is empty $\rightarrow$ True
isBalanced ("{}[]{}") $\rightarrow$ Balanced.

2) StackDemo.Generic.java

Field declaration

int max;

Object[] stack;

int top = -1;

Constructor 1 - no argument

StackDemo.Generic() {

 this(100);

}

this(100) - redirects execution to Constructor 2
Passing 100 as capacity.

Constructor 2 - with capacity.

StackDemo.Generic(int capacity) {

 max = capacity;

 stack = new Object[max];

 max = capacity.

 stack = new Object[max] - allocates an
 array of max empty.

Date _____
Page _____

```
new StackDemo_Generic < Integer > ();
```

1. call cons 1 run $\rightarrow$ calls this (100)

2. cons 2 run with capacity 100 $\rightarrow$ max=10,
Stack = new object [100]

max = 100, Stack = 100 num slots, top = -1

```
new StackDemo_Generic < > (6);
```

Cons 1 Skipped then cons 2 directly
run Capacity = 6, max = 6, stack = new
Object [6].

Push (T value)

```
void push (T value) {
```

```
    if (top == max - 1) {
```

```
        System.out.println ("stack overflow");
```

```
    } else {
```

```
        stack[++top] = value;
```

```
    }
```

```
}
```

is top already at the last valid index (max-1)?

If yes then print overflow, If No $\rightarrow$ ++top
store value there.

POP()

```
@SuppressWarnings ("unchecked")
```

```
T pop () {
```

```
    if (top == -1) {
```

```
        return null;
```

}

return (T) stack[top--];

}

Check empty first $top == -1 \rightarrow$ return Null
Otherwise read $stack[top]$, cast to T, then
decrement top

boolean isEmpty() {

return top == -1;

compares top to -1, no loop, O(1).

isMatchingPair (open, ~~char~~ close) ('{', '}')
Pure logic check

clause 1 :- open == '(' $\rightarrow$ '{' == '(' $\rightarrow$ false $\rightarrow$ clause 1 = false.

clause 2 :- open == '{' $\rightarrow$ '{' == '{' $\rightarrow$ true AND close ==
'}' $\rightarrow$ '}' == '}' $\rightarrow$ true $\rightarrow$ clause 2 = true

Clause 3 :- never evaluated
return true

isMatchingPair ('c', ']')

clause 1 :- open == 'c' $\rightarrow$ true, but close == ']' $\rightarrow$ false $\rightarrow$

clause 1 = false.

clause 2 $\rightarrow$ open = '{' $\rightarrow$ '(' == '{' $\rightarrow$ false $\rightarrow$
 clause 2 = false

clause 3 $\rightarrow$ open == '[' $\rightarrow$ '(' == '[' $\rightarrow$ false $\rightarrow$
 clause 3 = false.
 return false.

1) Integer stack Entered numbersStack = new
 StackDemo<Integer>()

operation	condition checked	stack	top	output.
Push (10)	top == 99? NO (top = -1)	[10]	0	-
Push (20)	top == 99? NO (top = 1)	[10, 20]	1	-
Pop ()	top == -1? NO	[10]	0	stack[1] = 20

2) ^{string} Integer stack Entered textStack = new
 StackDemo<String>()

Operations	Condition Checked	stack	top	output
Push("Hello")	top == 99? NO	[Hello]	0	-
Push("world")	top == 99? NO	[Hello world]	1	-
Pop ()	top == -1? NO	[Hello]	0	stack[1] = world

3) stackArrayIntDemo.java

capacity = 3

StackArrayIntDemo stack = new StackArrayIntDemo(3)

stack [,] top = -1

Choice	ch/ value	Operation	check	stack array	top	O/P
1	5	Push(5)	top == -1? NO (top = -1)	[5, ,]	0	5 pushed onto stack
2	15	Push(15)	top == -1? NO (top = 0)	[5, 15,]	1	15
3	-	Peek()	isEmpty? NO	[5, 15,]	1	TOP = 15
4	-	POP()	isEmpty? NO	[5, ,]	0	15 Popped
4	-	display()	isEmpty? NO loop i = top > 0	[5, ,]	0	stack Element (t to b) 5
5	-	exit	-	-	-	Exit

4) stack using LinkedList.java

- generic linked-list stack, main() hardcodes
push(10), push(20), push(30)

```
StackUsingLinkedList < Integer > stack = new Stack-  
usingLinkedList<>()
```

```
top = null size = 0
```

Operations	New code	Linked list	size
------------	-------------	-------------	------

Push(10)	Node(10)	10 → null	1
----------	----------	-----------	---

Push(20)	Node(20)	20 → 10 → null	2
----------	----------	----------------	---

Push(30)	Node(30)	30 → 20 → 10 → null	3
----------	----------	---------------------	---

display() → traverses from top ∴ stack elements
(top to bottom) : 30 20 10

peek() → isEmpty? no → returns top.data = 30
top elements : 30

pop() → value = top.data = 30 ; top = top.next
(node 20) ; size → returns 2

size() → size after pop : 2

display() → stack elements (top to bottom) : 20 10

5) StackUsingLinkedListIntDemo.java

choice	value	operat ⁿ	List(top→)	size	output
1	10	Push(10)	10	1	10 pushed onto stack
1	20	Push(20)	20→10	2	20 pushed onto stack
1	30	Push(30)	30→20→10	3	30 pushed onto stack
4	-	display()	30→20→10	3	Stack elements : 30 20 10
3	-	peek()	30→20→10	3	Top : 30
2	-	Pop()	20→10	2	30 Popped
5	-	Size()	20→10	2	Size: 2

6) ParenthesisMatchDemo.java

Expression = "(a+b)*c)"

calling :- isBalanced

Object :- stack < character > stack = new stack < > ()

loop :- for (i=0; i < expression.length(); i++)

i	ch	operation	stack	Result.
0	(	Push	(	-
1	a	not a bracket → no action	(	-
2	+	no action	(	-
3	b	no action	(	-
4	)	stack not empty → pop() → top = (; isMatchingPair('(', ')') → false	-	return false immediately

(a+b)*c → Not Balanced

{[()] } → Balanced

→ ReverseStringStackDemo.java

input = "CODE"

calling = reverse(input)

Stack < Character > stack = new Stack < > ()

for (i = 0 ; i < input.length() ; i++)

i	ch	operation	stack
---	----	-----------	-------

0	C	push	(C)
---	---	------	-----

1	O	push	(CO)
---	---	------	------

2	D	push	(COD)
---	---	------	-------

3	E	push	(CODE)
---	---	------	--------

loop end → stack ^{full} empty → loop stops

While loop - while (!stack.isEmpty())
 reversed.append(stack.pop())

operation	Popped Ch	stack after	reversed (sb)
pop	E	pop	E
pop	D	ED	ED
pop	C	EDC	EDC
pop	B	EDCB	EDCB
pop	A	EDCBA	EDCBA

8) Expression Evaluation Demo.java

expression = "2 3 1 * + 9 -"

evaluatePostfix(expression)

Stack <Integer> stack = new Stack <>();
 tokens = expression.split("\\s+");

["2", "3", "1", "*", "+", "9", "-"]

token	is operator?	operation	stack
2	No	Push(2)	2
3	No	Push(3)	2, 3
1	No	Push(1)	2, 3, 1
*	Yes	second = POP() = 1; first = POP() = 3; Push(3 * 1 = 3)	2, 3
+	Yes	second = POP() = 3; first = POP() = 2; Push(2 + 3 = 5)	5
9	No	Push(9)	5, 9
-	Yes	second = POP() = 9; first = POP() = 5; Push(9 - 5 = 4)	4

9 No · push(9) 5, 9
 - yes second = pop() = 9;
 first = pop() = 5;
 push(5 - 9 = -4) -4

Loop end → return: stack.pop() → -4
 evaluation postfix ("2 3 1 * + 9 -") → -4

9) UndooperationDemo.java

- interactive session,
Stack <String> actions = new Stack<>();
empty at start.

choices 1 ("Type Hello"), 1 ("Type World"), 2
 (undo), 3 (show)

choice	I/p	Operations	stack	output
1	"Type Hello"	push	Type Hello	Type Hello
1	"Type World"	push	Type Hello Type World	Type World
2	-	isEmpty? NO → pop()	Type Hello	undo
3	-	isEmpty? NO for each print	Type Hello	Current action Type Hello

* queue

1) QueueDemo.java

Fields

```
int max = 5;
```

```
int [] queue = new int [max];
```

```
int front = -1;
```

```
int rear = -1;
```

enqueue (int value)

```
if (rear == max - 1)
```

```
{ Print "Queue Overflow";
```

```
return;
```

```
}
```

```
if (front == -1) {
```

```
front = 0;
```

```
}
```

```
queue [++rear] = value;
```

```
Print value + " inserted into queue";
```

dequeue ()

```
if (isEmpty()) {
```

```
Print "Queue underflow";
```

```
return -1;
```

```
}
```

```
int removed = queue [front++];
```

```
if (front > rear)
```

```
{ front = -1; rear = -1;
```

```
}
```

return removed;

isEmpty() $\rightarrow$ return front = -1;

display() $\rightarrow$ "Queue is empty". Else loop for(
i = front; i <= rear; i++)
and Print queue[i].

main()

loop:- 1. Enqueue, 2. Dequeue, 3. Display, 4. Exit.

Choices:- 1(10), 2(20), 1(30), 3, 2, 2, 3, 1(40),
1(50), 1(60), 2, 2, 2, 2

Create object:- queueDemo = new QueueDemo()
queue = [0, 0, 0, 0, 0] front = -1, rear = -1

Step	choice	operation	checks	queue array	front	rear
1	1	enqueue(10)	rear == 4? No. front == 1? yes $\rightarrow$ front 0	[10, 0, 0, 0, 0]	0	0
2	1	enqueue(20)	rear == 4? No	[10, 20, 0, 0, 0]	0	1
3	1	enqueue(30)	rear == 4? No	[10, 20, 30, 0, 0]	0	2
4	3	display()	empty? no, i = 0, 2	[10, 20, 30, 0, 0]	0	2

5 2 dequeue() empty? No;
 removed = queue[0] = 10 [10, 20, 30, 0, 0] 1 2
 front → 1; Front → rear?
 1 > 2 No

6 2 dequeue() removed = queue[1] = 20 [10, 20, 30, 0, 0] 2 3
 front → 2; 2 > 2? No

7 3 display() i = 2 ... 2 [10, 20, 30, 0, 0] 2 3 4

8 1 enqueue(40) rear == 4? No [10, 20, 30, 40, 0] 2 3 4

9 1 enqueue(50) rear == 4? No [10, 20, 30, 40, 0] 2 4

10 1 enqueue(60) rear == 4? [10, 20, 30, 40, 50] 2 4
 Yes
 overflow

Q2) CircularQueueDemo.java

Fields

is full()

return (front == 0 & rear == max - 1) || (rear + 1 == front)

isEmpty()

dequeue()

display()

Entered choices:- enqueue 10, 20, 30, 40, 50, 60 →

De → De → En 60, 70, 80 → display →

De · X5 → De.

creating object :: queue = [0, 0, 0, 0] front = -1
rear = -1

Step Operation is Fullstack Pointer update queue

1. enqueue(10) (front == 0 & rear == 4)? isEmpty $\rightarrow$ front = 0, rear = 0
NO;
(rear + 1 == front)
0 == 1? NO

2. enqueue(20) NO; 1 == 0? NO rear++ $\rightarrow$ 1 [10, 20, 0, 0]

3. enqueue(30) NO rear $\rightarrow$ 2 [10, 20, 30, 0]

4. enqueue(40) NO rear $\rightarrow$ 3 [10, 20, 30, 40]

5. enqueue(50) NO; 4 == 0? NO rear $\rightarrow$ 4 [10, 20, 30, 40, 50]

6. enqueue(60) front == 0 & rear == 4? true - unchanged because overflow

7. dequeue() - removed = queue[0] = 10;
front == rear? NO [10, 20, 30, 40, 50]
front == 4? NO
 $\rightarrow$ front++

8. dequeue() - removed = 20;
front $\rightarrow$ 2 [10, 20, 30, 40, 50]

9. enqueue(60) (front == 0...) ? no, not empty; [60, 20, 30, 40, 50]
rear + 1 = 5 == front = 2? rear == max - 1 → rear = 0
No → Not full

10. enqueue(70) rear + 1 = 1 == 2? rear + 1
No [60, 70, 30, 40, 50]

11. enqueue(80) rear + 1 = 2 == front = 2 → true [60, 70, 30, 40, 50]
Overflow

12. dequeue() - removed = queue [0] = 10; Front == rear? no; Front == 4? no → Front++
to removed from circular queue

13. dequeue() - removed = 20, Front++ [30, 40, 50]

12. Display() - i = 2 print 30 → i = 3 print 40 → i = 4 print 50 → i = (4 + 1) % 5 = 0 print 60 → i = 1 print 70, i == rear → break
CQ = [30, 40, 50, 60, 70]

13. dequeue() - removed = 30; 30 removed

14. dequeue() - removed = 40; 40 removed

15 dequeue() - removed = 50;

50 removed

16 dequeue() - removed = queue[0]
= 60;

60 removed

17 dequeue() - removed = queue[1]
= 70;

70 removed

18 dequeue() isEmpty
yes

-

Circular
Queue
underflow

* ArrayBasics.java

Field :: Static int shifts; - a counter of how many elements moves happen.

insertAt(int[] a, int size, int index, int value)
- insert with shifting right.

deleteAt(int[] a, int size, int index) - delete with shifting left.

in main()

1) default values.

2) random access

3) insertAt(a, 4, 15) ; size++
 Loop:- i starts at size = 4, runs while i > 1

Step	i	Action	a(after)	shifts
start	-	-	[10, 20, 30, 40, 0, 0, 0, 0]	0
1	4	a[4] = a[3] (40)	[10, 20, 30, 40, 40, 0, 0, 0]	1
2	3	a[3] = a[2] (30)	[10, 20, 30, 30, 40, 0, 0, 0]	2
3	2	a[2] = a[1] (20)	[10, 20, 20, 30, 40, 0, 0, 0]	3
-	1	1 > 1 false	-	3
store	-	a[1] = 15	[10, 15, 20, 30, 40, 0, 0, 0]	3

4) deleteAt(a, 6, 0) ; size--

Step	i	Action	a(after)	shifts
Starts	-	-	[10, 15, 20, 30, 40, 50, 0, 0]	0
1	0	a[0] = a[1] (15)	[15, 15, 20, 30, 40, 50, 0, 0]	1
2	1	a[1] = a[2] (20)	[15, 20, 20, 30, 40, 50, 0, 0]	2
3	2	a[2] = a[3] (30)	[15, 20, 30, 30, 40, 50, 0, 0]	3
4	3	a[3] = a[4] (40)	[15, 20, 30, 40, 40, 50, 0, 0]	4
5	4	a[4] = a[5] (50)	[15, 20, 30, 40, 50, 50, 0, 0]	5
-	5	5 < 5 false → end	-	5
clear	-	a[size-1] = a[5] = 0	[15, 20, 30, 40, 50, 0, 0, 0]	5

4) bounds check : $q.length$ is 8, index = 8
 $a[8] = 99$ throws ~~Array~~ ~~Index~~ Out of Bounds Exception, ~~catch~~ caught by catch.

5) Jagged 2-D array

r	Row created	After edges = 1	Inner loop ($c = 1, \dots, r-1$)	Final row
0	$\text{new int}[1] \rightarrow$ [0]	[1]	$c = 1; 1 < 0$ False $\rightarrow$ no iterations	[0]
1	$\text{new int}[2] \rightarrow$ [0, 0]	[1, 1]	$c = 1; 1 < 1$ false $\rightarrow$ none	[1, 1]
2	$\text{new int}[3] \rightarrow$ [0, 0, 0]	[1, 0, 1]	$c = 1; t[2][1] =$ $t[1][0] + t[1][1] =$ $1 + 1 = 2$	[1, 2, 3]
3	$\text{new int}[4] \rightarrow$ [0, 0, 0, 0]	[1, 0, 0, 1]	$c = 1; [3][1] = t[2][0]$ $+ t[2][1] = 1 + 2 = 3;$ $c = 2; [3][2] = t[2][1]$ $+ t[2][2] = 2 + 1 = 3$	[1, 3, 3, 1]

"Jagged" = the rows have different lengths [1, 2, 3, 4]

6) growing an array :
 $\text{Arrays.copyOf}(a, a.length * 2)$

Access $a[i]$: $O(1)$

Insert / delete at the end : $O(1)$

In the middle or front : $O(n)$
growing : $O(n)$

* DynamicArray.java

Fields

Size vs capacity :-

Size , data.length

Constructor : `DynamicArray(int initialCapacity)`
`data = new Object[initialCapacity];`
`}`

• add(T item)

• grow()

• Capacity sequence starting at 4 :-

oldcap oldcap >> 1 oldcap + that oldcap + 1 newcap =

4	2	6	5	6
6	3	9	7	9
9	4	13	10	13
13	6	19	14	19
19	9	28	20	28

get / set / removeAt / toString :-

- get(index) :- IF index < 0 || index >= size
→ throws IndexOutOfBoundsException.

main() : 20 adds starting from capacity 4

Creating Object : list = new DynamicArray <>(4) →
data = [_, _, _, _], size = 0, capacity = 4

add #	Value	Size == data.length?	What happens	Size after	Capacity	Copies
1-4	10, 20, 30, 40	0, 1, 2, 3 vs 4 → NO	Plain store	4	4	0
5	50	4 == 4 → grow() 4 → 6	Copy 4 elements, then store	5	6	4
6	60	5 vs 6 → NO	Plain store	6	6	4
7	70	6 == 6 → grow() 6 → 9	Copy 6	7	9	10
8-9	80, 90	NO	Plain store	9	9	10
10	100	9 == 9 → grow() 9 → 13	Copy 9	10	13	19
11-13	110, 120, 130	NO	Plain store	13	13	19
14	140	13 == 13 → grow() 13 → 19	Copy 13	14	19	32
15-19	150, 160, 170, 180, 190	NO	Plain store	19	19	32
20	200	19 == 19 → grow() 19 → 28	Copy 19	20	28	51

main() : removeAt(0)

list.removeAt(0)

O/P :- removeAt(0) $\rightarrow$ 10 [20, 30, 40, ..., 200] Size =
capacity = 28

main() $\rightarrow$ amortised cost check.

$\sim O(1)$

- add : amortised $O(1)$
- get / set : $O(1)$
- removeAt : $O(n)$
- grow : $O(n)$